

# COS 350 Lecture 15

unix: crontab

interval timers

sigaction

signal behaviors

sigprocmask

CRONTAB(1)

CRONTAB(1)

NAME crontab - maintain crontab files for individual users

## SYNOPSIS

crontab file

crontab { -l | -r | -e }

## DESCRIPTION

Arranges jobs to be run at specified times.

## OPTIONS

- l Lists the crontab file for the invoking user.
- r Removes a user's crontab from the crontab directory.
- e Edit using the editor specified by the EDITOR environment variable.

A crontab file consists of lines of 5 numeric fields then a command.

The numeric fields are:

minute (0-59),

hour (0-23),

day of the month (1-31),

month of the year (1-12),

day of the week (0-6 with 0=Sunday).

Numeric fields may be: \* = all values, a single value, a comma separated list of values, or a range value1-value2.

**The numeric fields are:**

**minute (0-59),  
hour (0-23),  
day of the month (1-31),  
month of the year (1-12),  
day of the week (0-6 with 0=Sunday).**

**Numeric fields may be:**

**\* = all values  
a single value  
a comma separated list  
a range value1-value2**

### **Example 1: Cleaning up core files**

**This example cleans up core files every weekday at 3:15 am**

**15 3 \* \* 1-5 find \$HOME -name core 2>/dev/null | xargs rm -f**

### **Example 2: Mailing a birthday greeting**

**0 12 14 2 \* mailx john%Happy Birthday!**

### **Example 3: Reminder to go to class (jan-may, mon&wed at 5:15**

**15 17 \* 1-5 1,3 mailx houser%class in 15 minutes**

## last class: simple timers and signal handling

**alarm()** – schedule a SIGALRM signal in some number of seconds

**signal()** – specify a handler function for a signal

**pause()** – cause process to sleep until a signal is received

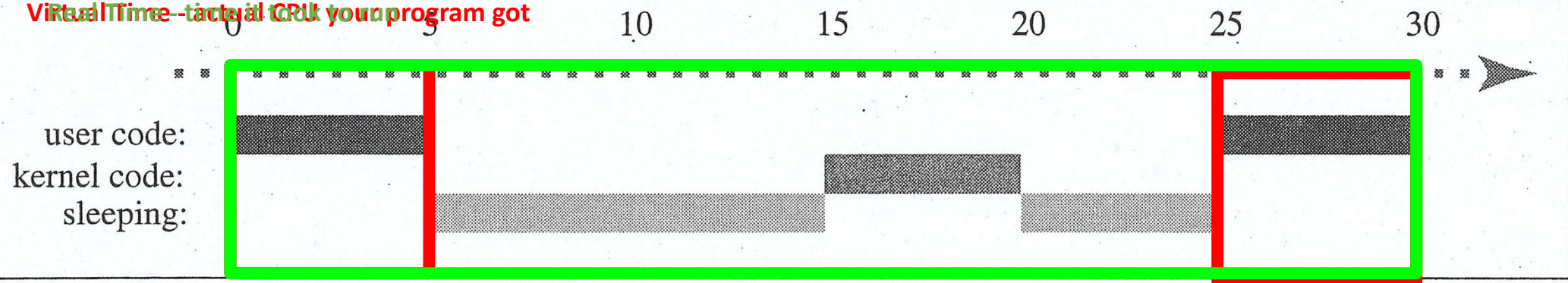
## this class: more sophisticated timers and signal handling

**setitimer()**, **getitimer()** – interval timers in microseconds

**sigaction()** – specify signal handler and behavior options

**sigprocmask()** – temporarily block signals

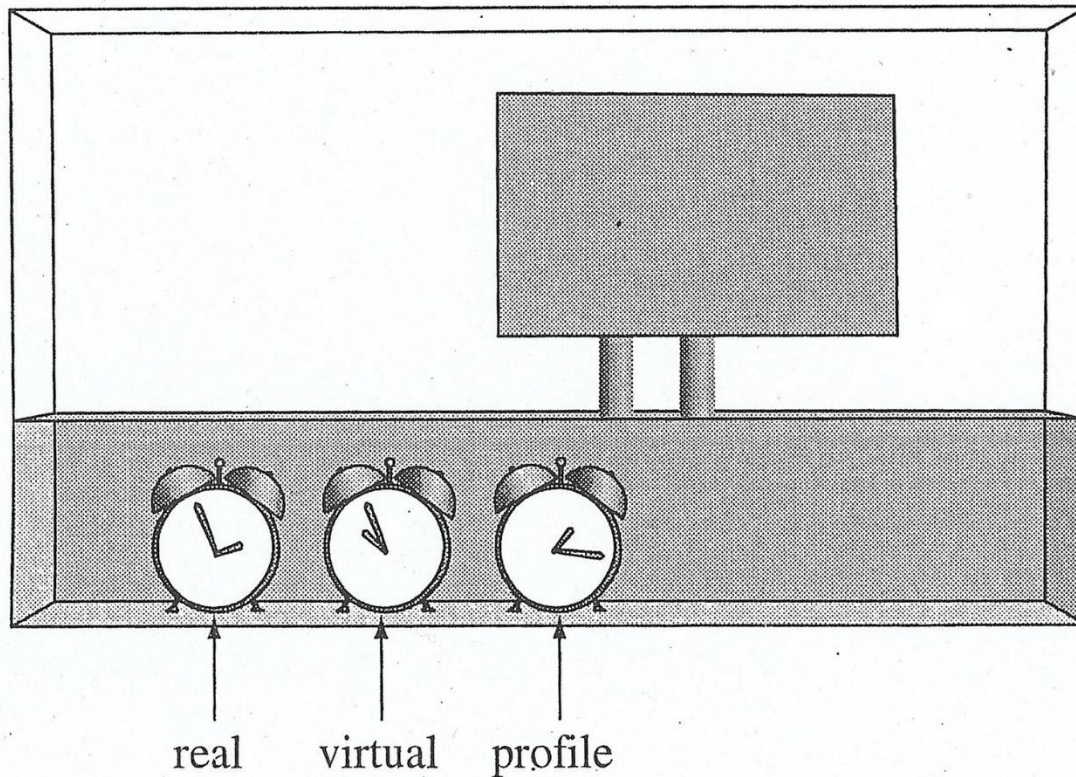
Virtual Time - actual CPU your program got



times: real 30 seconds

virtual 10 seconds  
(user mode)

prof 15 seconds  
(user+kernel mode)



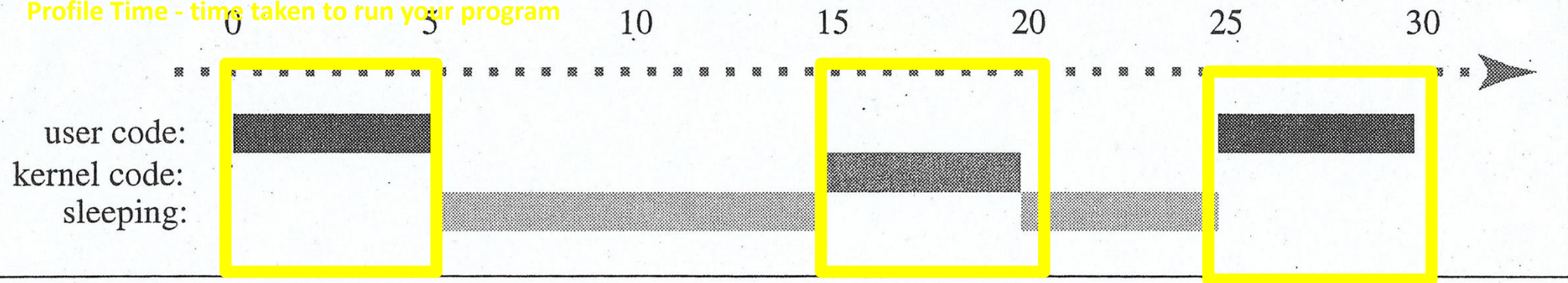
Every process  
has three timers.

Each timer has--  
two settings:  
the time until  
the first alarm  
and the interval  
between repeating  
alarms.

**Uses?**



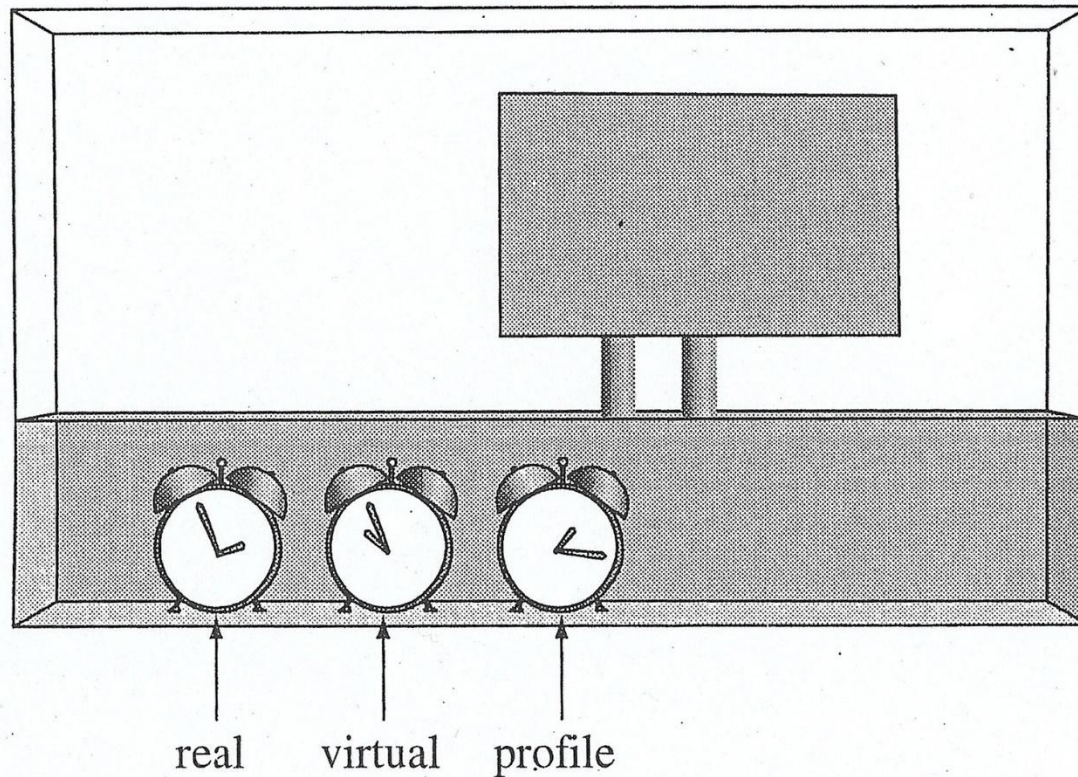
Profile Time - time taken to run your program



times: real 30 seconds

virtual 10 seconds  
(user mode)

prof 15 seconds  
(user+kernel mode)



Every process  
has three timers.

Each timer has--  
two settings:  
the time until  
the first alarm  
and the interval  
between repeating  
alarms.

**Uses?**

# Timer Uses?

- real time:
  - countdown timer
  - progress bar
- virtual time:
  - limit how much CPU you use, e.g. chess computer
  -
- profile time:
  - profiler to help with program performance

## NAME

getitimer, setitimer - get or set value of an interval timer

## SYNOPSIS

```
#include <sys/time.h>
```

```
int getitimer(int which, struct itimerval *curr_value);  
int setitimer(int which, const struct itimerval *new_value,  
              struct itimerval *old_value);
```

## DESCRIPTION

The system provides each process with three interval timers, each decrementing in a distinct time domain. When any timer expires, a signal is sent to the process, and the timer (potentially) restarts.

**ITIMER\_REAL** decrements in real time, and delivers **SIGALRM**.

**ITIMER\_VIRTUAL** decrements only when the process is executing, and delivers **SIGVTALRM** upon expiration.

**ITIMER\_PROF** decrements both when the process executes and when the system is executing on behalf of the process. **SIGPROF** is delivered upon expiration.

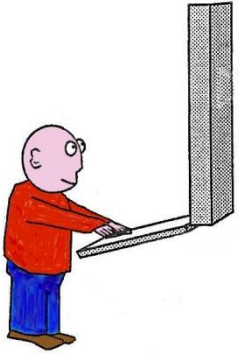


Timer values are defined by the following structures:

```
struct itimerval {  
    struct timeval it_value; /* current value */  
    struct timeval it_interval; /* next value */  
};
```

```
struct timeval {  
    long tv_sec; /* seconds */  
    long tv_usec; /* microseconds */  
};
```

|             | tv_sec | tv_usec |
|-------------|--------|---------|
| it_value    | 2      | 0       |
| it_interval | 0      | 750000  |



one234.c

# Complexity arises when you have multiple simultaneous signals

## Human signals

1. phone rings
2. knock on door
3. fire alarm

## Scenarios

- A. phone call interrupted by knock
- B. phone interrupted by phone
- C. knock interrupted by knock
- D. phone call interrupted by fire alarm
- E. fire alarm interrupted by phone call

## My Actions

- queue knock
- ignore 2<sup>nd</sup> phone call
- queue knock
- fire alarm has higher priority
- ignore phone call

## Limitation of the original signal() system call

1. You can't specify complex behaviors such as which other signals to block or accept.

2. Different implementations of unix/linux behaved differently

- On linux if SIGINT (Ctrl-C) arrives while in a handler for a previous SIGINT, it is queued.
  - On Sun machines the 1<sup>st</sup> SIGINT reset the handler, and then the 2<sup>nd</sup> SIGINT killed the process. (mousetrap mode)
- 
- On linux if a read() system call is interrupted, the read() is restarted after the signal handler finishes.
  - On Sun machines the read() was aborted and returned an error EINTR.
  - On both machines anything read from STDIN was discarded.



## NAME

sigaction

## SYNOPSIS

```
#include <signal.h>
```

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

## DESCRIPTION

The sigaction system call is used to change the action taken by a process on receipt of a specific signal.

signum     - specifies the signal  
act         - specifies the new action for the signal  
oldact     - if non-null, gets the previous action

The sigaction structure is defined as:

```
struct sigaction {  
    void (*sa_handler)(int sig);                // old style handler  
    void (*sa_sigaction)(int sig, siginfo_t *info, void *context); // new style  
    sigset_t sa_mask;  
    int sa_flags;  
}
```

## **sa\_mask**

specifies a mask of signals which should be blocked (i.e., added to the signal mask of the thread in which the signal handler is invoked) during execution of the signal handler. In addition, the signal which triggered the handler will be blocked, unless the SA\_NODEFER flag is used.

## **sa\_flags**

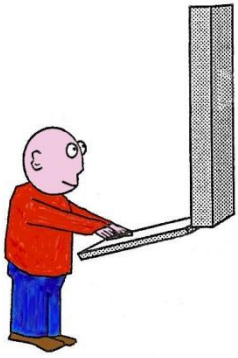
specifies a set of bit flags which modify the behavior of the signal handling process.

SA\_SIGINFO - Use the new style handler: sa\_sigaction

SA\_RESETHAND - Called signals are reset to default handler

SA\_NODEFER - Turn off auto blocking (allow nested calls)

SA\_RESTART - Restart interrupted slow system calls



sigactdemo.c

## NAME

sigprocmask - examine and change blocked signals

## SYNOPSIS

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

## DESCRIPTION

The sigprocmask call is used to change the list of currently blocked signals. The behavior of the call is dependent on the value of how, as follows.

**SIG\_BLOCK** - The set of blocked signals is the union of the current set and the set argument.

**SIG\_UNBLOCK** - The signals in set are removed from the current set of blocked signals. It is legal to attempt to unblock a signal which is not blocked.

**SIG\_SETMASK** - The set of blocked signals is set to the argument set.

If oldset is non-null, the previous value of the signal mask is stored in oldset.

## Example: Temporarily Blocking User Signals

A program can block SIGINT and SIGQUIT temporarily with the following code:

```
sigset_t  sigs, prevsigs;                /* define two signal sets */
sigemptyset( &sigs );                      /* turn off all bits      */
sigaddset( &sigs, SIGINT );                /* turn on SIGINT bit    */
sigaddset( &sigs, SIGQUIT );              /* turn on SIGQUIT bit   */
sigprocmask( SIG_BLOCK, &sigs, &prevsigs); /* add that to proc mask */

// .. modify data structure here ..

sigprocmask( SIG_SET, &prevsigs, NULL);    /* restore previous mask  */
```